GIK2Q3 Applied Big Data and Cloud Computing

Dalarna University

Reddit Big Data Pipeline

Project Report

Team B4

Meron Yuel Aarya

Navib Bajracharya

Pooja Khadka

Saba Anjum Na

Sweety Pandey

24th March 2026

Abstract

This report outlines the design, implementation and the deployment of a distributed big data pipeline for processing Reddit comment data on a large scale. The pipeline is regarding 12GB parquet file having the Reddit comments from the Hugging Face Pushshift Dataset. The system used Apache Spark (PySpark) for processing, Parquet for storage, and Docker with Kubernetes (Minikube) for deployment. A benchmark with Dask is included for performance comparison. Key stages include data ingestion from Hugging Face, validation through null checks and score filtering, transformation, and analysis via aggregations and window functions. Analytical results identified high-activity subreddits like AskReddit and revealed that smaller communities often yield higher average comment scores. This work demonstrates the effectiveness of combining modern data engineering frameworks with optimized storage formats to derive insights from massive public datasets.

1 Introduction

In today's data driven society, large amounts of user generated content are continuously generated through digital platforms. Social media, such as Reddit, is a rich data source where users share opinions, discuss current topics, and interact in various communities. This type of data offers opportunities for analyzing user behavior, popularity trends, and engagement levels. At the same time, the size and complexity of the dataset pose significant technical challenges in terms of storage, processing, and analysis. To effectively handle large scale datasets, distributed frameworks and optimized data processing methods are required.

The primary goal of this project is to design and implement a scalable big data pipeline for Reddit data. In this project, a scalable data pipeline based on Apache Spark is developed to analyze Reddit comments stored in Parquet format. The dataset consisted of over 80 million rows, making it suitable for investigating how modern big data tools perform in practical scenarios. This includes automating data collection, preprocessing and cleaning the data, storing it efficiently, and enabling meaningful analysis such as sentiment detection or trend identification. The project also aims to ensure that the pipeline is reliable, efficient, and capable of handling large volumes of data using appropriate big data tools and technologies.

The scope of this project is focused on building a functional pipeline that handles Reddit data from collection to analysis. It includes data ingestion, processing, storage and performing basic analytical tasks. However, the scope is limited to batch processing of historical data on local infrastructure; initial efforts were made to extend deployment to Azure cloud environments for improved scalability and production readiness. Real time analytics, advanced machine learning, and visualizations are outside the scope of the project. The focus is instead on building a robust, reproducible, and efficient data pipeline that can be scaled and further developed for more advanced use cases.

2 Data Description

This project uses a large scale public dataset of Reddit comments from Hugging Face. The data is stored in columnar Parquet format, enabling efficient storage and distributed processing. A subset of five Parquet files was used, totaling approximately 12 GB uncompressed. Data access is performed programmatically via the Hugging Face REST API that provided signed URLs for downloading each file shared.

Source	Format	Size	Rows	Access Method
Pushshift Reddit Comments (Hugging Face)	Parquet	12 GB	82,660,155	REST API

Table 1. Data sources used in this project.

The main dataset contains Reddit comments loaded from 5 datasets. Only a subset of relevant columns was used for processing to reduce memory overhead and improve performance.

Column Name	Data Type	Description
subreddit	String	Name of the subreddit where the comment was posted
created_utc	timestamp	Exact timestamp of the post creation
score	Integer	upvote score of the comment

Table 2. Schema of possessed dataset

3 Architecture

The project followed a modular system having end to end data pipeline architecture that is designed for scalability, high performance and reproducibility. It is composed of distinct stages consisting of data ingestion, validation, transformation, analysis, storage, and deployment. The data was first ingested from an external source and stored locally. Then, Apache Spark was used to perform the distributed processing across all the stages. The output of the pipeline was written into a Parquet Format. The entire pipeline was containerized using Docker and orchestrated as a Kubernetes Job within Minikube cluster, ensuring controlled execution, resource management, and repeatability. Logging and monitoring were included in every part of the pipeline to track performance and spot failures. This arrangement made it possible to debug, rerun, or extend each stage independently.

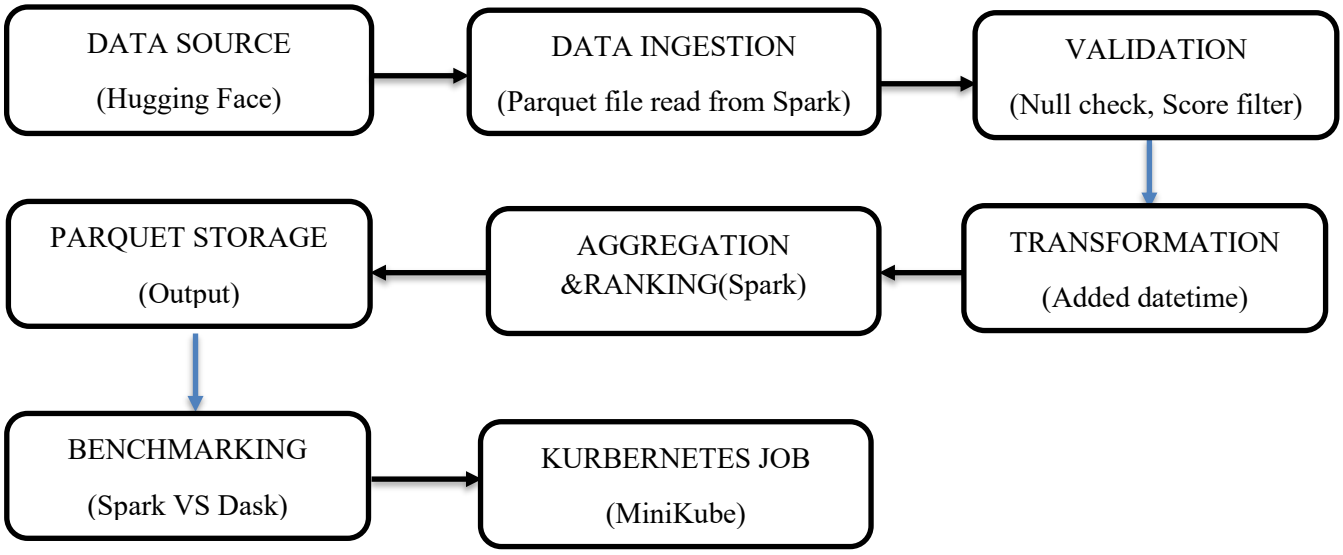


Figure 1. Reddit Big Data Pipeline Architecture

Technology	Role in System	Reason to choose
Apache Spark	Distributed data processing	High performance for large-scale batch workloads
Parquet	Data storage format	Columnar, compressed, efficient for analytics
Docker	Containerisation	Ensures consistent runtime environment
Kubernetes	Orchestration (via Minikube)	Manages job execution and resource allocation
Hugging Face API	Data source access	Provides scalable access to large public datasets
Dask	Benchmarking comparison	Evaluates alternative Python-based framework

Table 3. Technology choices and their roles.

We chose Spark over Dask because of its superior performance in aggregation of heavy workloads and advanced query optimization features which resulted in improving the execution efficiency. Dask offered faster starting times and had better Python libraries integration, but Spark provided more robust performance for large scale batch processing.

Parquet files were chosen over CSVs because of its columnar layout, built-in compression and schema support that reduced the storage size and improved reading performance.

Overall, this architecture enabled efficient, scalable batch processing while maintaining flexibility for future extensions. While the current implementation was run in a local Kubernetes environment using Minikube, initial efforts were made to deploy the pipeline to Microsoft Azure. This included exploring cloud-based container orchestration and storage services to improve scalability and move towards a production-ready setup.

4 Implementation

The primary ETL workflow of the Reddit Big Data Pipeline was majorly written in PySpark, with Dask being utilized in a subsequent benchmarking phase to compare framework performance on specific tasks. Reddit comment data was imported, cleaned, processed, analyzed, and stored using a systematic ETL technique. Every step was made to effectively manage big datasets while guaranteeing scalability and dependability.

4.1. Environment setup and pipeline initialization

Setting up the local execution environment was the first step in the pipeline. Environment variables like JAVA_HOME, HADOOP_HOME, and PySpark paths were specifically set as PySpark depends on Java and Hadoop. This guaranteed that Spark can function properly in a local Windows environment.

```
os.environ["JAVA_HOME"] = r"C:\Program Files\Eclipse Adoptium\jdk-
11.0.30.7-hotspot"
os.environ["HADOOP_HOME"] = r"C:\hadoop"
os.environ["PATH"] += os.pathsep + r"C:\hadoop\bin"
```

Next, a Spark session with all available cores, 8 GB of driver memory, and 50 shuffle partitions were created in local mode.

```
spark = SparkSession.builder \
    .appName("RedditBigDataPipeline") \
    .master("local[*]") \
    .config("spark.driver.memory", "8g") \
    .config("spark.sql.shuffle.partitions", "50") \
    .getOrCreate()
```

Setting the number of shuffle partitions enabled to optimize group-by operations, which were widely utilized later in the pipeline.

4.2. Data Ingestion

The pipeline started with Parquet Files ingestion from the Hugging Face dataset through the REST API. The API returned the signed URLs, which we used to download the data locally. After the local download, Spark read the files into the distributed Dataframe.

```
import requests

url = "https://huggingface.co/api/datasets/fddemarco/pushshift-reddit-
comments/parquet/default/train"
files = requests.get(url).json()

for i, file_url in enumerate(files[:5]):
    response = requests.get(file_url, stream=True)
    with open(f"data/{i}.parquet", "wb") as f:
        for chunk in response.iter_content(chunk_size=8192):
            f.write(chunk)
```

The dataset included the Reddit comments saved as Parquet files. These files were kept locally and were taken from the Pushshift dataset. Within ingestion, only the necessary fields were chosen rather than loading every column.

```
df = safe_read_parquet(
    files,
    select_cols=["subreddit", "score", "created_utc"]
)
```

Since Parquet files are in columnar format, Spark can be able to read only the columns that are required, which enhanced performance. Validation was another step in the ingestion process to ensure the dataset was not empty.

```
df = check_empty_dataframe(df, "main dataframe")
```

4.3. Data Validation and Cleaning

After data ingestion, the dataset was validated to ensure good data quality. We removed missing values from key columns and extremes outliers were filtered out.

```
df = df.dropna(subset=["subreddit", "score"])
df = df.filter(col("score") > -1000)
filtered_df = df.filter(col("score") > 0)
```

This stage guaranteed that the analysis was supported by reliable data. To monitor data quality, the pipeline additionally reported number of rows that were eliminated.

4.4. Data Transformation

In the transformation stage, raw fields were converted into usable format. By transforming timestamps into a legible format, transformation stage enhanced the dataset. UNIX timestamps were converted into human-readable datetime values, and only positively scored comments were retained for analysis.

```
df = df.withColumn("date", from_unixtime(col("created_utc")))
filtered_df = df.filter(col("score") > 0)
```

This conversion facilitated evaluation and made the data ready for future time-based analysis.

4.5. Aggregation and Analysis

There were two primary analytical tasks carried out by the pipeline.

Based on volume of comments, it first determined which subreddits were the most active:

```
top_subreddits = filtered_df.groupBy("subreddit") \
    .agg(count("*").alias("comment_count")) \
    .orderBy(col("comment_count").desc())
```

Secondly, it ranked each subreddit by calculating respective average scores:

```
avg_scores = df.groupBy("subreddit") \
    .agg(avg("score").alias("avg_score"))

window_spec = Window.orderBy(col("avg_score").desc())

ranked_subreddits = avg_scores.withColumn(
    "rank",
    rank().over(window_spec)
)
```

Ranking was made possible by using window function, that preserved aggregated data

structure. When combined, these analytics offered information about engagement quality and subreddit activity.

4.6. Storage

Parquet format was used to write the final findings back to disk:

```
top_subreddits.write.mode("overwrite").parquet("output/top_subreddits")
ranked_subreddits.write.mode("overwrite").parquet("output/ranked_subreddits")
```

Consistency was ensured and effective reuse in further processing or analysis was made possible by using the same format for output.

4.7. Benchmarking (Spark vs Dask)

To evaluate the performance and comparison, same operations were also implemented using Dask and then compared with Spark.

```
dask_df = dd.read_parquet("data/*.parquet")
dask_result = dask_df[dask_df["score"] > 0] \
.groupby("subreddit")["score"].mean().compute()
```

This comparison highlighted the trade-offs between Spark's optimization engine and Dask's lightweight execution.

4.8 Performance optimizations

Different optimization techniques were employed in the execution to enhance scalability and efficiency when handling big Reddit datasets.

Column pruning was done for reading only the necessary fields (such as subreddit, score, and date), column selection during ingestion minimized needless data loading. Spark reduced I/O operations and memory consumption by avoiding scanning unnecessary columns because Parquet allows columnar access.

Parquet's columnar format with built-in compression enabled faster reads and better storage efficiency than CSV, particularly benefiting analytical queries.

Additionally, shuffle partition tuning optimized Spark's data distribution during group-by and aggregation operations. The pipeline prevented excessive task overhead or underutilization of resources by setting the right number of shuffle divisions that resulted in more balanced parallel execution.

```
spark.conf.set("spark.sql.shuffle.partitions", "50")
```

Early data filtering was done before costly changes were executed as early filtering shrinks the dataset. Early removal of invalid or irrelevant records reduced the number of rows processed by following processes, speeding up execution.

When combined, these optimizations minimized computing time, lowered memory use, and guaranteed that the pipeline maintained its efficiency when scaling to huge datasets.

Overall, the pipeline adhered to a straightforward ETL workflow: data was taken from Parquet files, cleaned and verified, converted into a format that can be used, analyzed using aggregate and ranking, and then written back as output. Scalable processing was made possible with PySpark, and dependability was enhanced by extra logging and performance monitoring. By offering insights into different methods for distributed data processing, the Dask comparison enhanced implementation even more.

4.9 Deployment on Minikube with Dask

To evaluate distributed execution, the pipeline was deployed on a local Kubernetes cluster using MiniKube and a Dask setup. This deployment simulated a real cluster environment that included a scheduler and several worker nodes.

The cluster was first started and its status was verified.

```
minikube start --cpus=4 --memory=6144 --driver=docker
kubectl get nodes
```

Then Dask was run using Helm, which created one scheduler task and two worker pods

```
helm repo add dask https://helm.dask.org/
helm repo update

helm install my-dask dask/dask \
    --set scheduler.image.tag=2025.1.0 \
    --set worker.image.tag=2025.1.0 \
    --set scheduler.replicas=1 \
    --set worker.replicas=2 \
    --set jupyter.enabled=false
```

The state of the clusters was checked until all the pods were running. To allow the notebook file to communicate with the cluster, configuration of the port forwarding was done.

```
kubectl get pods -w
kubectl port-forward svc/my-dask-scheduler 8786:8786
kubectl port-forward svc/my-dask-scheduler 8787:80
```

Dask client was used to connect to the deployed scheduler.

```
deployed_client = Client("tcp://localhost:8786")
print(deployed_client)
```

Since Kubernetes pods could not directly access local files, the dataset was loaded locally and distributed to the cluster as a Dask DataFrame:

```
pandas_df = pd.read_parquet("data/0.parquet")
ddf_deployed = dd.from_pandas(pandas_df, npartitions=12)
```

We only loaded single parquet file for deployment instead of full dataset due to memory constraints in the Minikube environment, where worker nodes had limited RAM. When we attempted to load all 5 files (~12 GB), it resulted in worker failures and out-of-memory (OOM) errors. By reducing the input size, the pipeline could run reliably while still demonstrating distributed computation and task scheduling.

Additionally, the pipeline was then executed on the cluster using the distributed operations.

```
avg_score =d (
        ddf
        .query("score > 0")
        .groupby("subreddit")["score"]
        .mean()
        .nlargest(10)
        .compute())
```

Through this deployment, it showed how the same analytical workload can be executed in a distributed environment. However, performance was affected by the data serialization overhead because the dataset had to be transferred from the local machine to the worker pods. In a real cloud setup, this issue would be fixed by storing data in shared object storage that all workers could access.

5 Results

The pipeline processed 82,660,155 Reddit comments from 12 GB of Parquet files. The total execution time for the entire pipeline was 90.16 seconds, demonstrating that the solution can handle large data sets efficiently in a local Spark environment.

5.1 Data ingestion and validation

After data ingestion, the entire dataset was loaded correctly without errors. The initial load took 5.05 seconds and resulted in 82.6 million rows. After validation and filtering (removing null values and negative/extreme scores), the dataset was reduced to 74,962,882 rows, which corresponds to a reduction of 9.3%.

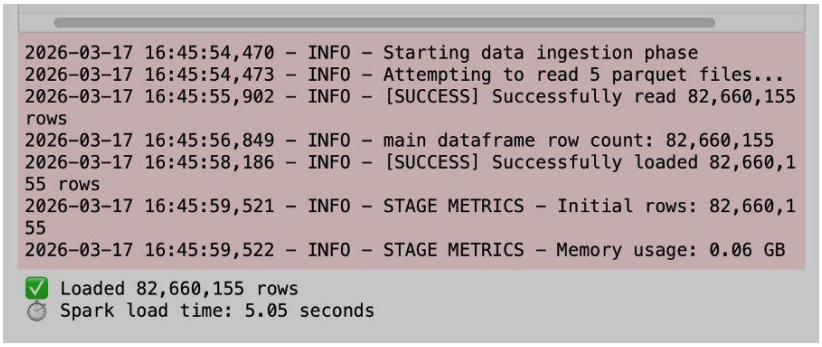


Figure 2. Data Ingestion Log output

This figure shows the log output from the data load step where all five Parquet files were loaded into Spark. A total of 82,660,155 rows were loaded in 5.05 seconds, indicating efficient I/O performance when using the column-based storage format.

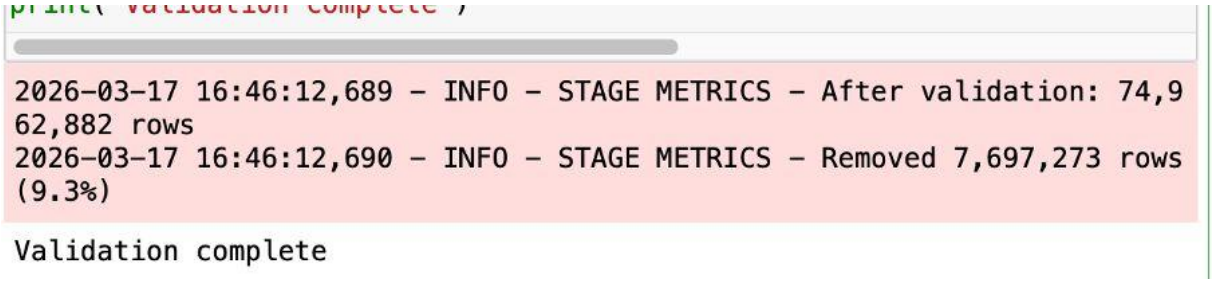


Figure 3. Data validation results

The figure illustrates the result after the validation step where invalid rows were filtered out. The dataset was reduced to 74,962,882 rows, which corresponds to a reduction of 9.3%, improving the data quality for further analysis. It showed that the pipeline not only loaded data efficiently but also ensured data quality before further analysis.

5.2 Aggregation: Most active subreddits

The first analysis identified the most active subreddits based on number of comments. The result showed that certain subreddits dominate the dataset.

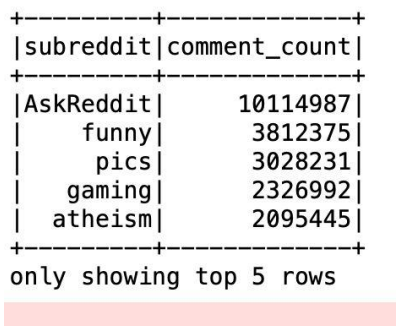


Figure 4. Aggregation Results – Top subreddits

The figure shows the most active subreddits based on number of comments. The results show that general subreddits, such as AskReddit, dominate the dataset with over 10 million comments, indicating high user activity in broad discussion forums. The top five subreddits were, AskReddit 10,114,987, funny 3,812,375, pics 3,028,231, gaming 2,326,992, and atheism 2,095,445.

5.3 Ranking based on average score

The second analysis calculated the average score per subreddit and ranked these using a window function.

```
+------------------+------------------+----+
|         subreddit|         avg_score|rank|
+------------------+------------------+----+
|            cognac|132.07692307692307|   1|
|         Bestof2011| 25.47137518684604|   2|
|  tvcharacterbattle|22.190594059405942|   3|
|forwardsfromgrandma|21.428571428571427|   4|
|      AlienBlueNews|              17.0|   5|
|            TeenR4R|              16.0|   6|
|             t:zero|15.910828025477707|   7|
| Shitty_Watercolour|15.137873754152825|   8|
|            Scatsex|              15.0|   9|
|        AlexisTexas|              15.0|   9|
+------------------+------------------+----+
only showing top 10 rows
```

Figure 5. Window function results – ranked subreddits

The figure shows the ranking of subreddits based on average score. The results indicate that smaller subreddits often had higher average engagement, suggesting that the quality of activity did not necessarily correlate with the volume of comments. Example of top results include cognac with average score of 132.08 and rank 1, Bestof2011 with average score of 25.47 with rank 2 etc.

5.4 Performance: Spark Vs Dask

A comparison between PySpark and Dask was conducted for three operations: data loading, filtering, and group-by aggregation.

Operation	Spark (s)	Dask (s)	Winner
Load Data	5.05	0.68	Dask
Filter	3.52	1.47	Dask
GroupBy	6.01	2.89	Dask

Figure 6. Spark vs Dask Performance Comparison

The figure demonstrates comparison of execution time between Spark and Dask for three operations. The results show that Dask was faster and it performed better in all tests in this local

environment, especially due to lower overhead. However, Spark offered better stability for complex pipelines and was more suitable for complex ETL processes. In the operations from above figure, we can observe that Dask performed faster on all three tasks.

5.5 Deployment on Kubernetes (Minikube)

The Reddit data pipeline was set up on a local Kubernetes cluster with Minikube and dask cluster deployment. This simulated a production-like container management environment. This setup shows how the pipeline could run in a separate, repeatable, and scalable way.

The deployment was carried out in several steps: first was to get the minikube cluster started with sufficient resource to handle dataset and verify that it was running and ready to use.

```
PS C:\WINDOWS\System32> minikube start --cpus=4 --memory=6144 --driver=docker
* minikube v1.38.1 on Microsoft Windows 11 Home Single Language 25H2
* Using the docker driver based on existing profile
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.50 ...
* Updating the running docker "minikube" container ...
* Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
* Verifying Kubernetes components...
  - Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Enabled addons: storage-provisioner, default-storageclass
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
PS C:\WINDOWS\System32> kubectl get nodes
NAME       STATUS   ROLES           AGE   VERSION
minikube   Ready    control-plane   14d   v1.35.1
```

Figure 7. Minikube cluster status

Next, the Dask cluster was deployed using Helm, creating one scheduler and two worker pods. The system was monitored using '*kubectl get pods'* until all components reached the Running state.

```
PS C:\WINDOWS\System32> kubectl get pods -w
NAME                                READY   STATUS    RESTARTS   AGE
my-dask-scheduler-bdd8df6b6-n67wc   1/1     Running   0          94s
my-dask-worker-5fc74c95ff-dwzww     1/1     Running   0          94s
my-dask-worker-5fc74c95ff-ggzfm     1/1     Running   0          94s
```

Figure 8. Dask Deployment on Kubernetes showing running pods

Within the notebook, a Dask client was connected to the deployed scheduler using the forwarded port. Since the worker pods could not directly access local files, the dataset was first loaded locally using pandas and then converted into a Dask DataFrame, which was distributed across the workers.



Figure 9. Cluster status Dashboard (CPU, Memory Usage)

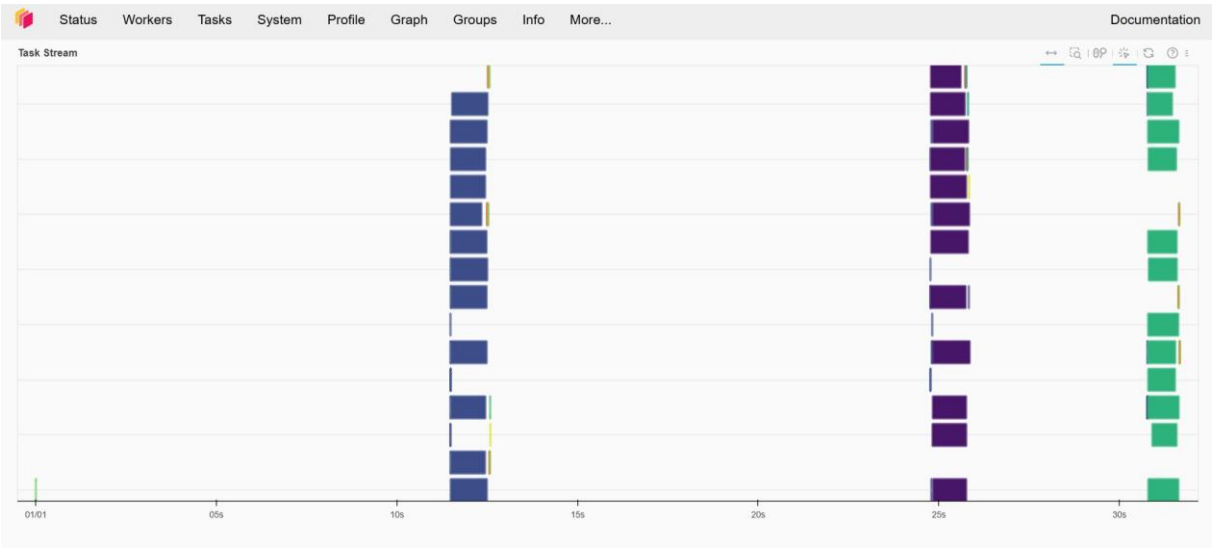


Figure 10. Task Stream Dashboard Status

Finally, the pipeline analysis (aggregation and transformations) was executed on the cluster, and performance was measured.

```
==================================================
 PERFORMANCE COMPARISON
==================================================
  Local  cluster (2 workers) : 9.0s
  Deployed cluster (Minikube): 34.1s
  Ratio                      : 3.79x
==================================================
```

Figure 11. Performance Comparison: Local vs Deployed Cluster

From the performance comparison, we witnessed that Dask cluster completed its execution with 9.0 seconds whereas the deployed minikube cluster took 31.1 seconds which was 3.79x times slower. The slower processing was because of data serialization and communication overhead between the local machine and kubernetes worker pods and not due to computational inefficiency.

6 Discussion and Conclusion

The project designed and deployed a distributed data pipeline for processing large-scale Reddit data using Apache Spark and Dask. Even though the pipeline executed correctly in both local and Minikube environments, the cluster deployed demonstrated slower performance because of data serialization and communication overhead, showing the limitations of running distributed system on a single machine. However, the implementation demonstrated key concepts such as parallel processing, task scheduling, and container orchestration using Kubernetes.

The performance was improved through optimization such as early data filtering, column selection, and the use of Parquet as a columnar storage format, although memory constraints required limiting the dataset size during deployment. Overall, the system delivered a scalable and reproducible pipeline, and provided a robust foundation for future work, particularly deployment on real distributed or cloud-based infrastructure where true performance benefits could be realized.

References

Apache Spark Documentation. (n.d.). Retrieved from Apache Spark: https://spark.apache.org/documentation.html

Authors, K. (n.d.). *Kubernetes Documentation*. Retrieved from Kubernetes: https://kubernetes.io/docs/tutorials/hello-minikube/

Dask Developers. (n.d.). *Dask*. Retrieved from Dask index: https://docs.dask.org/en/stable/index.html

fddemarco. (n.d.). *Hugging Face Pushshift-Reddit Dataset*. Retrieved from Hugging Face: https://huggingface.co/datasets/fddemarco/pushshift-reddit-comments

Microsoft. (n.d.). *Azure Functions Documentation*. Retrieved from Azure: https://learn.microsoft.com/en-us/azure/azure-functions/

Declaration

We confirm that this work is our own and that we have cited all used sources. Any use of AI tools (ChatGPT, GitHub Copilot, etc.) has been disclosed below.

AI Tools Used:

CHATGPT – For help in advance coding and text enhancement.

Claude AI – For help in trying to deploy data in AZURE Platform.

DeepSeek – For help in debugging code errors.

Signatures:

_____	_____
Member 1 — Meron Yuel Aarya	Member 2 — Navib Bajracharya
_____	_____
Member 3 — Pooja Khadka	Member 4 — Saba Anjum Na
_____	
Member 5 — Sweety Pandey	